Operazioni coi binari e conversioni

Raccolta Completa: 3 Lezioni ed Esercizi

Documento consolidato con testo integrale, tabelle, formule e tutti gli schemi visivi.
Formattato appositamente per l'ingestione e lo studio su Google NotebookLM.

✨ Fonte Unica per Google NotebookLM

🔢 Sistemi di Numerazione e Conversioni di Base

Nel mondo dell'informatica, i calcolatori elettronici non utilizzano la base decimale (in uso dagli esseri umani) per via della loro natura fisica: i circuiti integrati funzionano grazie a transistor che possono assumere solo due stati stabili di tensione (spento o acceso, assenza o presenza di corrente). Da qui nasce la necessità di padroneggiare il **sistema binario** e i suoi stretti parenti: l'**esadecimale** e l'**ottale**.

---

1. La Notazione Posizionale e il Teorema Fondamentale

Tutti i sistemi numerici moderni che utilizziamo sono **posizionali**: il valore espresso da una determinata cifra dipende dalla **posizione** che essa occupa all'interno del numero.

In una base generica b, un numero N formato dalle cifre $(c_{n-1}c_{n-2}\ldots c_1c_0, c_{-1}c_{-2}\ldots c_{-m})_b$ assume il valore decimale dato dal **Teorema Fondamentale della Numerazione Posizionale**:

$$N_{10} = \sum_{i=-m}^{n-1} c_i \cdot b^i = c_{n-1} \cdot b^{n-1} + \cdots + c_1 \cdot b^1 + c_0 \cdot b^0 + c_{-1} \cdot b^{-1} + c_{-2} \cdot b^{-2} + \ldots$$

Le Basi più Importanti in Informatica

Base	Nome	Simboli Utilizzati	Utilizzo Tipico
Base 10	**Decimale**	`0, 1, 2, 3, 4, 5, 6, 7, 8, 9`	Uso comune umano.
Base 2	**Binario**	`0, 1`	Linguaggio nativo delle CPU e dell'elettronica digitale.
Base 16	**Esadecimale**	`0, 1, ..., 9, A, B, C, D, E, F`	Indirizzi di memoria (RAM), codici colore web, MAC address.
Base 8	**Ottale**	`0, 1, 2, 3, 4, 5, 6, 7`	Permessi nei file system Linux/Unix (es. `chmod 755`).

> **Corrispondenza delle cifre esadecimali**
>
> Nel sistema esadecimale, non disponendo di singole cifre arabe per i valori da 10 a 15, si utilizzano le prime sei lettere dell'alfabeto:
>
> - **A** = 10
> - **B** = 11
> - **C** = 12
> - **D** = 13
> - **E** = 14
> - **F** = 15

---

2. Da Base Generica a Decimale (Sviluppo Polinomiale)

Per convertire un qualsiasi numero da una base b a decimale, basta applicare direttamente il principio dei pesi posizionali.

Da Binario a Decimale (Base 2 ➔ Base 10)

I pesi delle cifre da destra verso sinistra sono le potenze crescenti di 2:
$2^0 = 1$, $2^1 = 2$, $2^2 = 4$, $2^3 = 8$, $2^4 = 16$, $2^5 = 32$, $2^6 = 64$, $2^7 = 128$ ...

> **Esempio pratico (dal quaderno degli appunti)**
>
> Convertire il numero binario 1010112 in decimale:
>
> $1010112 = (1 \cdot 25) + (0 \cdot 24) + (1 \cdot 23) + (0 \cdot 22) + (1 \cdot 21) + (1 \cdot 20)$ 1010112
> $= 32 + 0 + 8 + 0 + 2 + 1 = 4310$

Da Esadecimale a Decimale (Base 16 ➔ Base 10)

I pesi sono le potenze di 16: $16^0 = 1$, $16^1 = 16$, $16^2 = 256$, $16^3 = 4096$ ...

> **Esempio pratico (dal quaderno degli appunti)**
>
> Convertire il valore esadecimale $01CF16$ in decimale:
>
> Ricordiamo che C = 12 e F = 15: $01CF16 = (0 \cdot 163) + (1 \cdot 162) + (12 \cdot 161) + (15 \cdot 160)$ $01CF16$
> $= 0 + 256 + 192 + 15 = 46310$

---

3. Da Decimale ad Altre Basi (Divisioni Successive)

Per convertire un numero intero decimale in una base b, si utilizza l'**algoritmo delle divisioni successive**:

1. Si divide il numero per la base b.
2. Si annota il quoziente intero e il **resto** ottenuto.
3. Si ripete la divisione del nuovo quoziente per b fino a quando il quoziente non diventa **0**.
4. **Attenzione**: il numero convertito si ottiene leggendo i resti **dal basso verso l'alto** (l'ultimo resto ottenuto corrisponde alla cifra più significativa, **MSB** - *Most Significant Bit*).

```
flowchart TD
    A["Numero Decimale N"] --> B["Dividi N per la base b"]
    B --> C["Calcola Quoziente Q e Resto R"]
    C --> D{"Quoziente Q = 0?"}
    D -- No --> E["N = Q, ripeti divisione"]
    E --> B
    D -- Sì --> F["Leggi tutti i resti R<br/>DAL BASSO VERSO L'ALTO (dall'ultimo al
```

Esempio 1: Da Decimale a Binario (43 ➔ Base 2)

Dividiamo ripetutamente per 2:

Divisione	Quoziente	Resto	Lettura
43 : 2	21	**1**	↑ (LSB - cifra meno significativa)
21 : 2	10	**1**	↑
10 : 2	5	**0**	↑
5 : 2	2	**1**	↑
2 : 2	1	**0**	↑
1 : 2	0	**1**	↑ (MSB - cifra più significativa)

Leggendo i resti dal basso verso l'alto: $43_{10} = 101011_2$

Esempio 2: Da Decimale a Esadecimale (463 ➔ Base 16)

Dividiamo ripetutamente per 16:

Divisione	Quoziente	Resto	Simbolo Esadecimale	Lettura
463 : 16	28	15	**F**	↑ (cifra meno significativa)
28 : 16	1	12	**C**	↑
1 : 16	0	1	**1**	↑ (cifra più significativa)

Leggendo i resti dal basso verso l'alto: $46310 = 1CF16$

---

4. Conversioni Rapide Dirette tra Binario, Esadecimale e Ottale

Poiché le basi 16 e 8 sono potenze esatte di 2 ($16 = 2^4$ e $8 = 2^3$), la conversione tra binario ed esadecimale (o ottale) **non richiede di passare per il decimale**!

Tabella di Equivalenza Universale

Decimale	Binario (4 bit)	Esadecimale	Ottale (3 bit)
0	`0000`	**0**	`000` (0)
1	`0001`	**1**	`001` (1)
2	`0010`	**2**	`010` (2)
3	`0011`	**3**	`011` (3)
4	`0100`	**4**	`100` (4)
5	`0101`	**5**	`101` (5)
6	`0110`	**6**	`110` (6)
7	`0111`	**7**	`111` (7)
8	`1000`	**8**	-
9	`1001`	**9**	-
10	`1010`	**A**	-
11	`1011`	**B**	-
12	`1100`	**C**	-
13	`1101`	**D**	-
14	`1110`	**E**	-
15	`1111`	**F**	-

Binario ➔ Esadecimale

1. Si raggruppano i bit a **quartetti (gruppi di 4 bit)** partendo da **destra verso sinistra**.

2. Se l'ultimo gruppo a sinistra non è completo, si aggiungono zeri in testa (*padding*).
3. Si traduce ciascun quartetto nella corrispondente cifra esadecimale.

> **Esempio**
>
> Convertire 110110111012 in esadecimale:
>
> 1. Raggruppiamo: `110` | `1101` | `1101`
> 2. Aggiungiamo lo zero a sinistra: `0110` | `1101` | `1101`
> 3. Traduciamo:
>   - `0110` = 6
>   - `1101` = D
>   - `1101` = D
>
> Risultato: $6DD16$

Esadecimale ➔ Binario

Si sostituisce semplicemente ogni singola cifra esadecimale con il suo esatto quartetto di 4 bit binari:

- $A716 \rightarrow A = 1010, 7 = 0111 \implies 101001112$

---

5. Numeri Frazionari e Reali (con la Virgola)

Cosa succede quando abbiamo un numero non intero, come ad esempio $5,14$? Il numero viene separato in due parti distinte:

1. **Parte Intera**: si converte con le consuete divisioni successive per 2.
2. **Parte Frazionaria**: si converte con l'algoritmo delle **moltiplicazioni successive per 2**.

Algoritmo delle Moltiplicazioni Successive per la Parte Frazionaria

1. Si prende la sola parte decimale (es. $0,14$).
2. Si moltiplica per 2.

3. La **parte intera** del prodotto (0 oppure 1) rappresenta la prima cifra binaria dopo la virgola.
4. Si prende la nuova parte frazionaria e si ripete la moltiplicazione per 2.
5. Il procedimento si arresta quando la parte decimale diventa 0 (conversione esatta) oppure quando si raggiunge il numero desiderato di bit di precisione (approssimazione) o quando si individua un periodo.
6. **Attenzione**: le cifre binarie della parte frazionaria si leggono **DALL'ALTO VERSO IL BASSO**!

```
[PARTE FRAZIONARIA]  * 2  =  [INTERO].[NUOVA FRAZIONE]
                                    |
                                    v
                          Bit letto DALL'ALTO
```

Esempio Pratico Guidato: Conversione di 5, 1410 in Binario

> **Dagli appunti del quaderno: 5, 1410**
>
> **Fase 1: Parte Intera (510)**
>
> - 5 : 2 = 2 con resto **1**
> - 2 : 2 = 1 con resto **0**
> - 1 : 2 = 0 con resto **1**
> - Leggendo dal basso: 510 = 1012
>
> **Fase 2: Parte Decimale (0, 1410)** Moltiplichiamo per 2 prendendo la parte intera:

Moltiplicazione	Risultato	Cifra Intera (Bit)	Lettura
$0,14 \cdot 2$	$0,28$	**0**	↓ (primo bit dopo virgola)
$0,28 \cdot 2$	$0,56$	**0**	↓
$0,56 \cdot 2$	$1,12$	**1**	↓ (si tiene $0,12$ per il passo successivo)
$0,12 \cdot 2$	$0,24$	**0**	↓
$0,24 \cdot 2$	$0,48$	**0**	↓
$0,48 \cdot 2$	$0,96$	**0**	↓
$0,96 \cdot 2$	$1,92$	**1**	↓

Leggendo le cifre dall'alto verso il basso: $0,1410 \approx 0,0010001\ldots_2$

Unendo parte intera e frazionaria: $5,1410 \approx 101,00102$ (approssimato a 4 cifre frazionarie)

Da Binario con la Virgola a Decimale

Per tornare da un binario con la virgola al decimale, si usano le potenze negative di 2 per le cifre dopo la virgola:

- $2^{-1} = \frac{1}{2} = 0,5$
- $2^{-2} = \frac{1}{4} = 0,25$
- $2^{-3} = \frac{1}{8} = 0,125$
- $2^{-4} = \frac{1}{16} = 0,0625$

Verifica inversa di $101,00102$:

> $101,00102 = (1 \cdot 22) + (0 \cdot 21) + (1 \cdot 20) + (0 \cdot 2{-}1) + (0 \cdot 2{-}2) + (1 \cdot 2{-}3) + (0 \cdot 2{-}4)$
> $= 4 + 0 + 1 + 0 + 0 + 0,125 + 0 = 5,12510$ *(La differenza rispetto a 5,14 è dovuta al troncamento a 4 bit; aumentando i bit l'approssimazione diventa sempre più accurata).*

🎯 Scheda Riassuntiva Finale

```mermaid
flowchart LR
    DEC["Decimale 10"]
    BIN["Binario 2"]
    HEX["Esadecimale 16"]

    DEC -- "Divisioni per 2 (dal basso in alto)" --> BIN
    BIN -- "Somma potenze di 2" --> DEC

    DEC -- "Divisioni per 16 (dal basso in alto)" --> HEX
    HEX -- "Somma potenze di 16" --> DEC

    BIN -- "Gruppi di 4 bit" --> HEX
    HEX -- "Espansione a 4 bit" --> BIN
```

➕ Operazioni Aritmetiche Binarie

L'aritmetica binaria segue esattamente le stesse regole logiche dell'aritmetica decimale tradizionale che impariamo alla scuola primaria. L'unica differenza sostanziale risiede nella **base**: avendo a disposizione solo due cifre (`0` e `1`), il "riporto" (*carry*) o il "prestito" (*borrow*) scattano non appena si raggiunge o si supera il valore **2** (che in binario si scrive 102).

---

1. L'Addizione Binaria

L'addizione tra due bit segue una tabella di verità elementare:

Bit A	Bit B	Somma (S)	Riporto (C - Carry)	Spiegazione Decimale
`0`	`0`	**0**	`0`	0 + 0 = 0
`0`	`1`	**1**	`0`	0 + 1 = 1
`1`	`0`	**1**	`0`	1 + 0 = 1
`1`	`1`	**0**	**1**	$1 + 1 = 210 = 102$ (scrivo 0 e riporto 1)

> **Caso con Riporto Precedente**
>
> Se alla colonna si aggiunge anche un riporto proveniente dalla colonna precedente (1 + 1 + 1): $1 + 1 + 1 = 310 = 112 \implies$ scrivo 1 e riporto 1

Esempio 1: Addizione a Due Operandi

Calcoliamo $1101_2 + 1011_2$ ($13_{10} + 11_{10} = 24_{10}$):

```
  ¹ ¹ ¹          <- Riporti
    1 1 0 1   +  (13)
    1 0 1 1   =  (11)
  ---------
  1 1 0 0 0      (24)
```

Verifica in decimale: 16 + 8 = 24. Corretto!

Esempio 2: Addizione a Tre Operandi (dal quaderno degli appunti)

Calcoliamo la somma: 10102 + 1012 + 111002

Convertiamo prima in decimale per avere un riscontro:

- 10102 = 8 + 2 = 1010
- 1012 = 4 + 1 = 510
- 111002 = 16 + 8 + 4 = 2810
- Totale atteso: 10 + 5 + 28 = 4310

Incolonnamento ed esecuzione con gestione dei riporti multipli:

```
  ¹ ¹ ¹ ¹        <- Riporti generati
      1 0 1 0 +    (10)
        1 0 1 +    (5)
    1 1 1 0 0 =    (28)
  -----------
    1 0 1 0 1 1    (43)
```

Analisi colonna per colonna (da destra a sinistra):

1. **Colonna 0**: 0 + 1 + 0 = 1 (riporto 0).
2. **Colonna 1**: 1 + 0 + 0 = 1 (riporto 0).
3. **Colonna 2**: 0 + 1 + 1 = 210 = 102 $\implies$ 0 con **riporto 1**.
4. **Colonna 3**: riporto 1 + 1 + 0 + 1 = 310 = 112 $\implies$ 1 con **riporto 1**.
5. **Colonna 4**: riporto 1 + 1 = 210 = 102 $\implies$ 0 con **riporto 1**.
6. **Colonna 5**: riporto 1 $\implies$ 1.

Risultato finale: $1010112 = 32 + 8 + 2 + 1 = 4310$. Perfetto!

---

2. La Sottrazione Binaria Diretta (con Prestito)

La sottrazione segue le regole:

Bit A	Bit B	Differenza (D)	Prestito (*Borrow*)	Spiegazione
`0`	`0`	**0**	`0`	0 – 0 = 0
`1`	`0`	**1**	`0`	1 – 0 = 1
`1`	`1`	**0**	`0`	1 – 1 = 0
`0`	`1`	**1**	**1**	Si chiede un prestito di 1 alla colonna a sinistra (102 – 1 = 210 – 1 = 1)

Esempio 1: Sottrazione con Prestito Semplice (dal quaderno)

Calcoliamo $110112 – 1102$ ($2710 – 610 = 2110$):

```
  -1          <- Prestito chiesto dalla colonna 2
  1 1 0 1 1 -   (27)
      1 1 0 =   (6)
  ---------
  1 0 1 0 1     (21)
```

Passaggi dettagliati:

1. 1 – 0 = 1
2. 1 – 1 = 0
3. 0 – 1: non si può fare. Si chiede in prestito 1 alla colonna a sinistra. Lo 0 diventa $102 = 210$. Quindi 2 – 1 = 1.

4. La cifra a sinistra che ha prestato (era 1) diventa 0. Quindi 0 – 0 = 0.
5. L'ultimo 1 a sinistra scende: 1.

Risultato: $101012 = 16 + 4 + 1 = 2110$.

Esempio 2: Sottrazione con Cascata di Prestiti (dal quaderno)

Calcoliamo $110102 - 112$ ($2610 - 310 = 2310$):

```
  -1        <- Prestito
1 1 0 1 0 -   (26)
      1 1 =   (3)
---------
1 0 1 1 1     (23)
```

Passaggi:

1. 0 – 1: chiede prestito al bit adiacente. Diventa $102 - 1 = 1$.
2. Il bit che ha prestato diventa 0. Ora abbiamo 0 – 1: deve chiedere a sua volta prestito a sinistra. Diventa $102 - 1 = 1$.
3. Il bit della colonna successiva (era 0, che a sua volta ha propagato il prestito da 1) gestisce la differenza: scendono 1, poi 0, poi 1.
4. Risultato: $101112 = 16 + 4 + 2 + 1 = 2310$.

> **Come fanno i computer reali?**
>
> I circuiti delle CPU raramente eseguono la sottrazione diretta con prestito, perché richiederebbe circuiti complessi (*Full Subtractor*). I calcolatori trasformano invece la sottrazione in un'**addizione con il Complemento a 2**: $A - B = A + (-B)$. Questo permette di usare lo stesso identico circuito sommatore! *(Vedi lezione successiva)*.

---

3. La Moltiplicazione Binaria (Shift-and-Add)

La moltiplicazione binaria è incredibilmente semplice rispetto a quella decimale perché la tavola pitagorica ha solo 4 combinazioni:

- $0 \cdot 0 = 0$
- $0 \cdot 1 = 0$
- $1 \cdot 0 = 0$
- $1 \cdot 1 = 1$

Ogni riga intermedia del prodotto o è una sequenza di zeri (se il moltiplicatore ha `0`) oppure è la copia identica del moltiplicando (se il moltiplicatore ha `1`), traslata a sinistra di una posizione (*shift logico*).

Esempio Guidato (dal quaderno degli appunti)

Calcoliamo $10102 \cdot 1012$ ($1010 \cdot 510 = 5010$):

```
    1 0 1 0  *   (10)
      1 0 1  =   (5)
-----------
    1 0 1 0      <- 1010 * 1 (shift 0)
  0 0 0 0        <- 1010 * 0 (shift 1)
1 0 1 0          <- 1010 * 1 (shift 2)
-----------
1 1 0 0 1 0      (50)
```

Verifica in base 10: $1100102 = (1 \cdot 25) + (1 \cdot 24) + (0 \cdot 23) + (0 \cdot 22) + (1 \cdot 21) + (0 \cdot 20)$
$= 32 + 16 + 0 + 0 + 2 + 0 = 5010$Il risultato è esatto al 100%!

---

4. La Divisione Binaria tra Interi

La divisione binaria ricalca il metodo della divisione euclidea a due cifre:

1. Si confronta il divisore con le prime cifre a sinistra del dividendo.
2. Se il divisore “ci sta” (è minore o uguale), la cifra del quoziente è **1** e si sottrae il divisore.

3. Se non ci sta, la cifra del quoziente è **0**.
4. Si abbassa la cifra successiva del dividendo e si prosegue fino all'esaurimento dei bit.

Esempio Guidato (dal quaderno degli appunti)

Calcoliamo la divisione 110112 : 112 (2710 : 310 = 910 con resto 0):

```
  1 1 0 1 1  | 1 1
- 1 1        +---------
  ---        | 1 0 0 1   <- Quoziente
  0 0 0
    - 0 0
    -----
      0 1
      - 0 0
      -----
        1 1
      - 1 1
      -----
        0 0              <- Resto finale = 0
```

Spiegazione passo-passo:

1. Prendiamo le prime due cifre del dividendo: 112. Il divisore 112 ci sta **1** volta. Scriviamo `1` nel quoziente e calcoliamo 11 – 11 = 00.
2. Abbassiamo la terza cifra (`0`): abbiamo 00. Il divisore 11 ci sta **0** volte. Scriviamo `0` nel quoziente.
3. Abbassiamo la quarta cifra (`1`): abbiamo 01. Il divisore 11 ci sta **0** volte. Scriviamo `0` nel quoziente.
4. Abbassiamo la quinta cifra (`1`): abbiamo 11. Il divisore 11 ci sta **1** volta. Scriviamo `1` nel quoziente e calcoliamo 11 – 11 = 00.
5. Non ci sono più cifre da abbassare. Il resto è **0**.

Verifica in decimale:

- Quoziente: 10012 = 8 + 1 = 910
- Resto: 0
- 9 · 3 = 27. Il calcolo coincide perfettamente!

Numeri con Segno e Complemento a 2

All'interno di un computer non possiamo inserire il carattere grafico `+` o `-` per indicare se un numero è positivo o negativo: la memoria fisica è composta esclusivamente da celle binarie che possono contenere solo `0` o `1`.

Per memorizzare ed elaborare i numeri interi relativi (Z) occorre quindi adottare una **convenzione di codifica**. Nel corso della storia dell'informatica sono stati ideati tre metodi principali, fino ad arrivare allo standard universale adottato oggi da ogni microprocessore: il **Complemento a 2**.

---

1. Modulo e Segno (MSB come Bit di Segno)

Il metodo più intuitivo consiste nel dedicare il bit più significativo (**MSB** - *Most Significant Bit*, il primo a sinistra) all'indicazione del segno:

- `0` indica segno **positivo (+)**
- `1` indica segno **negativo (-)**
- I restanti N – 1 bit indicano il **valore assoluto (modulo)** del numero.

```
flowchart LR
    subgraph Registro a 8 Bit
        S["Bit 7: SEGNO<br/>(0 = +, 1 = -)"] --- D["Bit 6 .. 0: MODULO<br/>(Valore a
    end
    style S fill:#ffcdd2,stroke:#d32f2f,stroke-width:2px;
    style D fill:#e1f5fe,stroke:#0288d1,stroke-width:2px;
```

> **Esempio su 8 bit:**
>
> - +510 = 000001012
> - −510 = 100001012

I Due Gravi Difetti del Modulo e Segno:

1. **Presenza del "Doppio Zero"**:
   - `00000000` = +0
   - `10000000` = −0 Questo spreca una configurazione e costringe i circuiti logici a raddoppiare i controlli di uguaglianza per verificare se un valore è nullo.
2. **Circuiti aritmetici complessi**: non è possibile sommare direttamente due numeri in modulo e segno con un semplice sommatore elettronico (es. (+5) + (−5) = 00000101 + 10000101 = 10001010 = −10, risultato palesemente errato!).

---

2. Complemento a 1 (C1)

Nel **Complemento a 1**, i numeri positivi mantengono la normale rappresentazione binaria con `0` all'MSB. Per ottenere il corrispondente numero negativo, si applica l'operatore logico **NOT**: **si invertono semplicemente tutti i bit** (ogni `0` diventa `1` e ogni `1` diventa `0`).

Matematicamente, per un registro a N bit, equivale a sottrarre il valore da una maschera di tutti `1` ($2^N - 1$).

> **Esempio pratico (dal quaderno degli appunti)**
>
> Rappresentare +90 e −90 su 8 bit in Complemento a 1:
>
> 1. Scriviamo +9010 in binario a 8 bit: $90_{10} = 01011010_2$
> 2. Calcoliamo −90 sottraendo da 11111111_2 (oppure invertendo ogni bit):

```
  1 1 1 1 1 1 1 1 -
  0 1 0 1 1 0 1 0 =   (+90)
-----------------
  1 0 1 0 0 1 0 1     (-90 in C1)
```

Limite del Complemento a 1:

Anche il Complemento a 1 soffre del difetto del **doppio zero**:

- `00000000` = +0
- `11111111` = −0

---

3. Complemento a 2 (C2): La Soluzione Perfetta

Il **Complemento a 2** risolve brillantemente ogni problema ed è il sistema utilizzato in tutte le architetture moderne (x86, ARM, RISC-V).

Regola Fondamentale di Calcolo

Per trovare l'opposto di un numero in Complemento a 2: C2(X) = C1(X) + 1

> **In parole semplici**
>
> 1. **Inverti tutti i bit** del numero (scambia gli `0` con gli `1`).
> 2. **Aggiungi 1** al risultato (somma binaria con eventuale propagazione del riporto).

Esempio 1: Calcolo di −90 su 8 bit (dal quaderno degli appunti)

1. Valore positivo: +9010 = 010110102
2. Complemento a 1 (invertiamo i bit): 101001012
3. Aggiungiamo 1:

```
  1 0 1 0 0 1 0 1 +
                1 =
----------------
  1 0 1 0 0 1 1 0
```

4. Risultato: −9010 in Complemento a 2 su 8 bit è 101001102.

Metodo Rapido (Scorciatoia Visiva)

Esiste un trucco pratico immediato per fare il Complemento a 2 senza calcolare somme a parte:

1. Parti da **destra verso sinistra**.
2. **Copia inalterati tutti i bit** fino a quando incontri il **primo `1` compreso**.
3. Da quel punto in poi verso sinistra, **inverti tutti i restanti bit**.

Verifica su +90:

```
Valore:       0  1  0  1  0  [0  1  0]   (il primo '1' da destra è al bit 1)
                                ^
Copia:                       [0  1  0]   (invariati fino al primo '1')
Inverti:      1  0  1  0  0
Risultato:    1  0  1  0  0   0  1  0  -> 10100110_2 (Identico!)
```

---

4. Il Peso del Bit di Segno in C2

Un modo straordinario e profondo per comprendere il Complemento a 2 è osservare i **pesi posizionali**. In un numero a N bit in C2, **tutti i bit mantengono i loro consueti pesi positivi tranne il bit più significativo (MSB), che assume peso NEGATIVO**:

Peso dell'MSB = -2^{N-1}

Su un byte (8 bit):

Bit 7 (MSB)	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0 (LSB)
−128	+64	+32	+16	+8	+4	+2	+1

> **Verifica del valore −90 = 10100110$_2$:**

> Valore = (−128 · 1) + (64 · 0) + (32 · 1) + (16 · 0) + (8 · 0) + (4 · 1) + (2 · 1) + (1 · 0)
> Valore = −128 + 32 + 4 + 2 = −128 + 38 = −9010! La formula funziona con assoluta precisione matematica!

---

5. Come Funziona la Sottrazione con C2: A − B = A + (−B)

Il vantaggio epocale del Complemento a 2 è che **elimina la necessità di costruire circuiti sottrattori**: la sottrazione viene eseguita come una normale somma!

A − B = A + C2(B)

> **Regola del Riporto Finale (Carry-Out)**
>
> Quando eseguiamo una sottrazione trasformata in addizione in C2, se si genera un **riporto uscente oltre la dimensione del registro (MSB)**, questo riporto va **ignorato e scartato**!

Esempio Guidato: Calcolo di 5 − 3 (dal quaderno degli appunti)

Lavoriamo su **4 bit** (i numeri rappresentabili vanno da −8 a +7):

- A = +510 = 01012
- B = +310 = 00112

Passo 1: Calcoliamo il complemento a 2 di B (cioè −3)

- Invertiamo i bit di `0011`: `1100`
- Aggiungiamo 1: `1100 + 1 = 1101_2` (−310)

Passo 2: Eseguiamo la somma 5 + (−3)

```
   ¹ ¹ ¹          <- Riporti
     0 1 0 1 +    (+5)
     1 1 0 1 =    (-3 in C2)
    ---------
  (1) 0 0 1 0     (+2)
   ^
   |--- Riporto finale (Carry-out): SI ELIMINA!
```

Scartando il riporto finale di overflow sulla quinta posizione, il risultato rimasto nel registro a 4 bit è: 00102 = +210Il calcolo 5 − 3 = 2 è esatto!

---

6. Intervallo di Rappresentabilità e il Fenomeno dell'Overflow

Con un registro di N bit in Complemento a 2:

- Esiste **un solo zero**: 0000 … 0 = 0.
- I numeri positivi vanno da 0 a $2^{N-1} - 1$.
- I numeri negativi vanno da -1 a -2^{N-1}.

Range C2 : $[-2^{N-1}, \quad +2^{N-1} - 1]$

Numero di Bit (N)	Valore Minimo (Negativo)	Valore Massimo (Positivo)
4 bit	$-2^3 = -8$	$+2^3 - 1 = +7$
8 bit (Byte)	$-2^7 = -128$	$+2^7 - 1 = +127$
16 bit (Short)	$-2^{15} = -32.768$	$+2^{15} - 1 = +32.767$
32 bit (Int)	$-2^{31} \approx -2,14 \cdot 10^9$	$+2^{31} - 1 \approx +2,14 \cdot 10^9$

Che cos'è l'Overflow (Trabocco)?

L’**overflow** si verifica quando il risultato di un’operazione aritmetica **eccede l’intervallo rappresentabile** con il numero di bit a disposizione.

> **Come riconoscere l'Overflow?**
>
> L’overflow può avvenire **soltanto** quando si sommano due numeri con lo **stesso segno** (entrambi positivi o entrambi negativi):
>
> 1. **Positivo + Positivo = Negativo** $\implies$ **OVERFLOW!** (es. su 8 bit: +100 + +50 = +150, ma il massimo è +127; il risultato diventa erroneamente un numero negativo).
> 2. **Negativo + Negativo = Positivo** $\implies$ **OVERFLOW!**
>
> *Nei circuiti delle CPU, la condizione di overflow viene segnalata impostando l’apposito flag hardware V (Overflow Flag) nel registro di stato.*